Гигиена Git

Английский оригинал: GIT_HYGIENE.md.

Справка по соглашениям для веток, хуков и коммитов. Всё здесь либо уже применяется в репозитории, либо включается одним вызовом `git config` на клон.

Ветки

`main` — ветка интеграции, только слияние: изменение попадает в неё через pull request, никогда прямым push. Работа идёт в ветках `agentN/<тема>`.

Правила уборки, по которым сокращался набор веток:

- ветка, чей конец является предком `main`, слита и её можно удалить;
- ветка с коммитами, которых нет в `main`, сохраняется, пока не решён её pull request.

```
git fetch --all --prune
for b in $(git for-each-ref --format='%(refname:short)' refs/remotes/origin/); do
    git merge-base --is-ancestor "$b" origin/main && echo "merged: $b"
done
```

.gitignore

Файл игнорирования закрывает используемые окружения и инструменты: `.venv/`, `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `build/`, `dist/`, редакторные следы и локальные черновые каталоги. Файлы захватов (`*.pcap`, `*.pcapng`) игнорируются по умолчанию, а фикстуры, корпус и эталонные выгрузки возвращаются явно.

Крупные собранные бинарники не попадают в историю: `presentation/video.mp4` игнорируется и пересобирается из `presentation/VIDEO.md`. Закоммиченные артефакты (`slides.pdf`, `slide.pptx`, `screencast.mp4`) — намеренные результаты.

.gitattributes

Бинарные ресурсы помечены как `binary`, чтобы Git никогда не пытался делать по ним текстовый diff или построчное слияние, а `*.pcap`/`*.pcapng` — как данные, а не текст. Вывод сборки сворачивается в однострочную сводку. Текстовые файлы нормализуются к LF при выгрузке.

Хук pre-commit

`.githooks/pre-commit` линтит подготовленные Python-файлы стартовым набором правил и запускает быстрые тесты протокола и гипотез. Включается один раз на клон:

```
git config core.hooksPath .githooks
```

Обойти один коммит можно через `git commit --no-verify`. Хук аккуратно пропускается, если нет Python с `ruff`, поэтому он никогда не блокирует машину без дополнений для разработки.

Сообщения коммитов

Коммиты следуют Conventional Commits, на английском, тема не длиннее 72 символов. Шаблон включается один раз на клон:

```
git config commit.template .gitmessage
```

Используемые типы: `feat`, `fix`, `docs`, `test`, `chore`, `refactor`, `style`, `perf`, `build`, `ci`, `revert`. Коммиты слияний и небольшое число ранних записей истории предшествуют соглашению и оставлены нетронутыми; переписывание истории вне области.

Секреты

`git log --all -p` не находит ни токена доступа, ни ключа, ни пароля. Строки `ghp_`/`github_pat_` встречаются только внутри документации самой этой проверки, как шаблон поиска, никогда как значение. `.env.example` несёт только заполнители; `.env` игнорируется. Рабочий токен читается из переменной окружения `GITHUB_TOKEN` и никогда не записывается в файл.

CHANGELOG

`CHANGELOG.md` пишется из `git log`, сгруппирован по итерациям, с короткими хешами коммитов, внёсших каждое изменение.

.mailmap

`.mailmap` сворачивает исторические написания автора в GitHub-хэндл сопровождающего, поэтому `git shortlog` и `git log` относят каждый коммит к одной личности.